

Executive Summary

Equity estimation via Monte Carlo is a trade-off between accuracy and latency. Too few samples yields noisy equity estimates, risking mis-decisions; too many wastes time or causes timeouts. Empirical evidence suggests that *very* high precision ($SE \ll 0.1\%$) is often unnecessary in obvious spots, but poor precision near decision thresholds can cost EV. For example, even a 1% equity error on a 5-BB pot is only $\sim 0.05BB$, but consistent errors can add up and flip marginal calls or folds. In practice, reducing Monte Carlo SE is known to tighten confidence intervals and improve EV estimates ¹, so sample budgeting should aim to reduce SE enough that the 95% CI around an equity estimate is small relative to decision margins. For instance, some equity engines target $SE \sim 0.002\%$ (0.002% equity error) before stopping ², a level far beyond what's needed for most decisions but illustrative of high-precision calibration.

Given a strict 10s per-action cap (with 9s effectively usable for sampling), we recommend an **adaptive hybrid approach**: use a modest fixed baseline of samples (or time) to get a rough equity, then selectively extend sampling for close calls until a confidence criterion or time limit is reached. Specifically, we suggest: (1) a *fast pre-evaluation* path (e.g. a few hundred-thousand sims or a cheap heuristic) to identify obviously strong/weak hands; (2) if the equity result is borderline (within a tolerance of the decision threshold), run additional Monte Carlo sampling up to a time budget, stopping when the confidence interval on equity is sufficiently narrow; (3) log key metrics (equity, SE, samples, time) for each decision to verify accuracy. This balances decisional robustness with latency. Three concrete options are contrasted below.

Options Comparison

(A) Fixed-Quota Monte Carlo: Always run a fixed number of samples (e.g. 5K–10K) for each postflop equity/discard calculation. *Pros*: simple, predictable; no adaptive logic needed. *Cons*: either under-samples tough spots (high variance) or wastes time on trivial spots. In practice, 5K–10K sims take only ~ 0.002 – 0.01 s if inner equity calls are fast, but can still miss subtle equity differences; OMPEval's examples show 5–10k sims giving ~ 0.5 – 1% equity SE. This might be too coarse for marginal raise/fold decisions.

(B) Time-Budgeted Adaptive Sampling: Allocate a fixed time budget (e.g. 0.05–0.5s) for equity estimation, but stop early if an error tolerance is reached. For example, one can use a sequential stopping rule: after each batch of ~ 100 – 500 samples, compute the current equity and standard error, and halt when the half-width of the 95% CI is below a threshold (or when the best discard choice is clear by a large margin). This approach ensures latency limits (never exceeds the cap) and focuses effort where needed. For instance, the OMPEval library example stops as soon as standard error falls below 0.002% or after 5s ². In our context, a looser criterion (say $SE \lesssim 0.5$ – 1%) and a 0.05s–0.5s cap per call would be more reasonable. *Pros*: automatically spends more time on hard cases; avoids wasted cycles on easy cases. *Cons*: more complex to implement (need confidence-tracking), risk of overhead.

(C) Hybrid Precomputed + Monte Carlo: Precompute or cache equity estimates for frequent scenarios, then refine with light Monte Carlo at runtime. For example, pre-calculate equity of all 2-card hole combinations against typical ranges for common boards (or use enumerations/isomorphisms), and store

results in a table. At runtime, use a quick table lookup as a baseline and then run a brief Monte Carlo (≤ 1000 samples) to adjust for the exact range or betting context. Deep learning could also be an element (inspired by DeepStack): use a fast neural “intuition” network to approximate equity/counterfactual values, then correct via a few Monte Carlo draws. Caching variants (e.g. recognizing suit isomorphisms) can greatly reduce needed sims ³. *Pros*: can achieve very fast average performance, leveraging offline computation. *Cons*: large storage or training needed; less flexible with new ranges; risk of stale precomputed values if opponents’ ranges change.

Each approach has tradeoffs. Fixed sampling is easiest but inflexible, time-budgeted is robust but more engineering, and hybrid can be very efficient but requires maintenance of auxiliary data. In any case, variance-reduction methods should be used where possible: OMPEval’s solver for Omaha reports a $\sim 3\times$ speedup by exploiting suit isomorphisms ³. Similarly, ACPC evaluators used “duplicate” sampling (antithetic deals) to cut variance dramatically ¹. In practice, caching common computations (e.g. partial run-outs) and stratified sampling (e.g. separating cases by board texture) can reduce samples needed by factors of 2–10 in some cases.

Recommended Approach

We recommend a **time-limited adaptive Monte Carlo** strategy with conservative default quotas and strong observability. Concretely: for postflop (flop/turn) equity and discard decisions, run Monte Carlo with an initial small sample (e.g. 500–2000 sims). Compute the equity (or best discard option) and its standard error. If the equity is far above or below the decision threshold (by, say, $>3\times$ the current SE), stop immediately – the choice is clear. If it is near the threshold, continue sampling in batches (e.g. 500 samples) until either (a) the confidence interval no longer overlaps the threshold, (b) a hard time budget (e.g. 0.05–0.1s) is reached, or (c) we reach an aggressive sample cap (e.g. 10k). The exact budget can scale by street/context: allow more time on flop (more unknown cards) than on turn, and none on river (equity is then exact). For multi-player or discard evaluations, multiply accordingly (e.g. 3 possible discards may need up to $\sim 3\times$ the samples of a single equity call).

Justification: This meets latency limits (never overruns 10s per action) while striving for “good-enough” precision. It mirrors how top bots operate at millisecond scales. For example, the AlphaHoldem bot took only $\sim 2.9\text{ms}$ per decision (on a GPU) ⁴, implying that investing tens of milliseconds per Monte Carlo is safe under a 10s budget for most real-time competitions. In ACPC settings, most bots used sub-10ms per move and relied on heavy offline precomputation and abstraction; our 9–50ms (0.009–0.05s) target is relatively generous by comparison. The early-stop logic ensures that “obvious” spots (e.g. pocket rockets vs super-wide range) incur minimal delay, while tough spots get extra attention. Logging SE and remaining uncertainty allows us to be confident decisions are near-optimal: as noted, lower Monte Carlo SE directly translates to tighter confidence bounds on EV ¹.

We also recommend **three guardrails**: (1) enforce a strict hard cap (e.g. exit after 9s or 10k samples) to prevent timeouts; (2) fall back on conservative heuristic if sampling fails (e.g. assume worst-case equity or default to quick rule); (3) gradual rollout of any changes. For example, if a new sampling strategy is enabled, track its overall EV against the old method to detect regressions. Observability (see below) is key to tune these parameters.

Implementation Considerations

- **Observability:** Log every equity/discard call's outcome, including the estimated equity, SE/CI, number of samples used, and time taken. Tag logs with context (street, pot size, ranges, discard choice) so we can analyze performance. Watching sample counts and SE over many hands will reveal if the bot is under-sampling (e.g. very high SE in key decisions). A "health metric" could be the fraction of decisions where the CI spans zero profit margin.
- **Variance Reduction Techniques:** Implement known speedups: exploit suit symmetry and card isomorphisms (as in OMPEval) to speed up both simulation and caching ³. If feasible, use stratified or antithetic sampling: e.g. pairing deals or forcing each player's share of deck to balance high/low outcomes. Use a fast fixed enumerator (like PokerStove's core engine) for trivial subcases (e.g. heads-up, or final card). Consider reusing samples across similar scenarios (e.g. identical ranges at different nodes, if legal).
- **Adaptive Logic:** Use a statistical stopping rule. A simple approach is to compute the one-sided Wald confidence interval after each batch and stop when the lower (or upper) bound has crossed a folding or betting threshold. To avoid bias, ensure stops are based only on already-seen data. (Sequential test theory can guide safe stopping rules.) In code, check a breakpoint (every 50-100 sims) to honor time budget. OMPEval's pattern – callback every 0.25s – is instructive ².
- **Performance Budgeting:** Distribute the 10s across possible future calls. Since later streets have fewer decisions, we can "save" time earlier. For example, spend up to 0.05-0.2s on flop decisions, but only ~0.01-0.05s on turns. Housekeep worst-case budgets (e.g. abort at 9s total). Threading: if equity calls are multi-threaded, watch total CPU use so as not to starve other tasks. Non-deterministic thread scheduling should still respect the time cap.
- **Opponent Modeling Interaction:** If the bot maintains opponent ranges or beliefs, adapt sample size accordingly. Narrower assumed ranges reduce outcome variance, so fewer sims might suffice. Conversely, highly uncertain ranges might warrant more sampling or broader use of worst-case assumptions. (Any opponent modeling should integrate into the simulation seed logic to remain deterministic if needed.) If using exploitative (non-GTO) adjustments, ensure Monte Carlo sim respects that adjusted strategy in generated hands.
- **Security & Privacy:** Ensure random seeds are handled securely: do not reuse sequences in ways an adversary could predict. If logging equity calculations or opponent modeling data, safeguard privacy (encrypt logs or avoid logging live opponent data). Use well-tested RNG libraries (e.g. C++ `<random>` with fixed seed for reproducibility in testing). If using third-party libs (pkrbot/PokerStove), audit them for known vulnerabilities and ensure thread-safety. (PokerStove's code is old; confirm it supports aborting or time-limited use, or else use a modern fork.)
- **Compatibility/Risks:** External equity engines (PokerStove, OpenPoker, etc.) may not support incremental abort or may use global state. Test that any chosen library works under tight timeouts and that its results are consistent across runs (for debugging). Watch for library version mismatches (an update to PokerStove might change random sampling behavior). Finally, if our bot is required to

be deterministic in certain contexts (e.g. for repro or fairness), fix RNG seeds per decision, or log seeds.

- **Failure Modes & Mitigation:** Under-sampling can cause “flip-flop” decisions: e.g., raising when folding is slightly better. To mitigate, the confidence-stop logic will reduce the probability of misclassification. If we detect oscillation (e.g. half of runs fold, half call), fall back to the safer action (usually folding is less costly). Also monitor key performance metrics (win-rate per street) to catch regressions. In production, implement a *rollback mechanism*: if a new sampling policy degrades the bot’s EV in testing or live play, revert to the previous safe policy or reduce its aggressiveness. One tactic is to A/B test: deploy adaptive sampling only part-time while keeping a control, then compare outcomes.
- **Performance/Testing:** Benchmark the sampling code in isolation. E.g., replicate environments like ACPC testbeds and measure how many sims you get per millisecond. Ensure integration with main loop respects the overall bot timing (the rest of decision logic + Monte Carlo must fit in 10s). Use profiling to identify any slow equity calculations; replacing those with faster (but equivalent) methods can greatly multiply throughput.

Bibliography / References

- Davidson, J. (2014). *The Baseline Approach to Agent Evaluation*. Univ. of Alberta. (Discusses Monte Carlo SE vs variance reduction ¹.)
- Zhao, E., et al. (AAAI 2022). *AlphaHoldem: AI for Heads-Up No-Limit Poker*. (Reports 2.9ms per decision for top bot ⁴.)
- Zekyll (2022). *OMPEval: Fast Poker Equity Calculator* (GitHub). (Example code showing stopping at $SE \approx 0.002\%$ and $3\times$ speedup using board isomorphisms ² ³.)
- HRC (HoldemResources) *Monte Carlo Sampling Guide*. (Industry documentation on convergence criteria and sampling, e.g. $CI < 5$ as a practical target.)
- Billings, D. & Kan, M. (ICGA Journal). *A Tool for Direct Assessment of Poker Decisions (DIVAT)*. (Discusses EV noise and assessment; relevant for understanding decision precision.)

¹ poker.cs.ualberta.ca

<https://poker.cs.ualberta.ca/publications/jdavidson.msc.pdf>

² ³ [GitHub - zekyll/OMPEval: Fast C++ poker hand evaluator and equity calculator](https://github.com/zekyll/OMPEval)

<https://github.com/zekyll/OMPEval>

⁴ [AlphaHoldem: High-Performance Artificial Intelligence for Heads-Up No-Limit Poker via End-to-End Reinforcement Learning](https://cdn.aaai.org/ojs/20394/20394-13-24407-1-2-20220628.pdf)

<https://cdn.aaai.org/ojs/20394/20394-13-24407-1-2-20220628.pdf>